SISTEMA DE GESTIÓN DOCUMENTAL

Manual técnico

Arquitectura, modelo documental, API REST y operación

Versión 1.0 · Octubre de 2026
Proyecto académico · CUNOC

1. Introducción

Este manual describe la implementación técnica del **Sistema de Gestión Documental (DMS)**, una aplicación web para almacenar, organizar, compartir y auditar archivos `.txt` y `.md` entre los usuarios de una organización.

El documento está dirigido a quienes necesitan instalar, mantener, evaluar o extender el sistema: desarrolladores, administradores de base de datos, docentes y revisores técnicos. Se explican la arquitectura, el modelo documental, las reglas de negocio, la API, los mecanismos de seguridad, las decisiones de rendimiento y los procedimientos de operación.

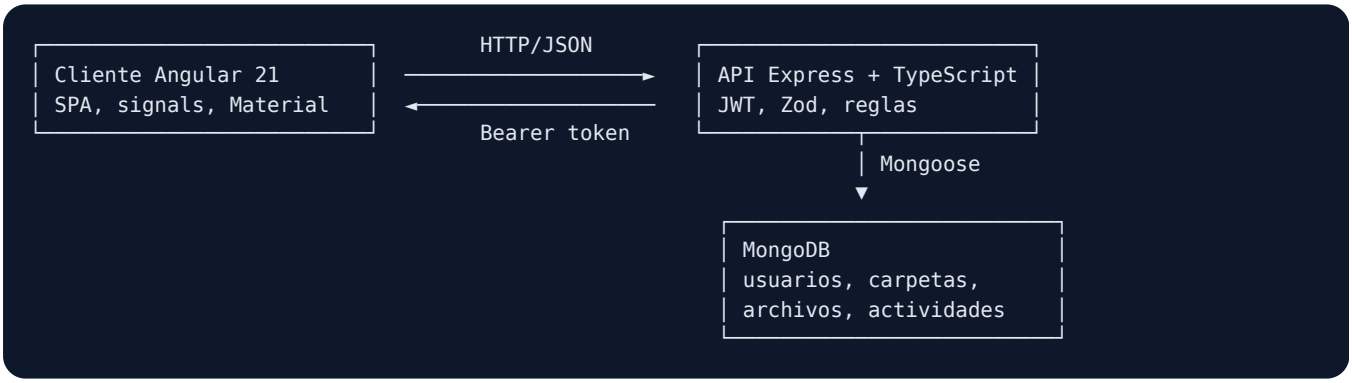
El repositorio se divide en dos aplicaciones independientes:

- `backend/`: API REST con Node.js, Express 5, TypeScript, Zod y Mongoose.
- `frontend/`: aplicación SPA con Angular 21 standalone, signals y Angular Material.

Ambas se comunican por HTTP/JSON. En desarrollo, el servidor de Angular redirige `/api` al backend mediante `proxy.conf.json`.

2. Arquitectura general

2.1 Vista de alto nivel



El cliente no accede directamente a MongoDB. Toda operación pasa por la API, que valida el token, comprueba permisos y registra la actividad correspondiente.

2.2 Capas del backend

Cada módulo respeta el siguiente flujo:

```
Routes → Middlewares → Controller → Service → Repository → Model
                          |            |
                          |            ├── Validators (reglas de negocio)
                          |            ├── Permisos (autorización)
                          |            └── Servicios auxiliares (compartidos, papelera)
                          └── Traducción HTTP de resultados y errores
```

- **Routes:** declara endpoints y aplica autenticación, autorización y validación Zod.

- **Controller:** extrae datos de `req`, invoca el servicio y responde con el código HTTP.
- **Service:** orquesta el caso de uso, aplica reglas y registra actividad.
- **Repository:** encapsula consultas y agregaciones de MongoDB.
- **Model:** define esquemas, índices, valores por defecto y transformaciones JSON.
- **Validators:** concentran reglas reutilizables de unicidad y jerarquía.
- **Servicios de permisos:** resuelven acceso directo y heredado.
- **Servicios de compartidos y papelera:** separan operaciones con efectos masivos.

La inyección de dependencias es manual y centralizada en `backend/src/shared/container/container.ts`.

2.3 Capas del frontend

```
app.routes.ts
  └── Layout protegido (sidebar + topbar)
        ├── Mi Unidad → Explorador reutilizable → Menú contextual
        ├── Visor / Editor
        ├── Papelera
        ├── Perfil
        └── Administración (dashboard, usuarios, reportes, actividad)
```

- Componentes **standalone**, sin `NgModule`.
- Estado local con **signals** y detección de cambios **zoneless**.
- Rutas **lazy** con `loadComponent`.
- `authGuard`, `guestGuard` y `adminGuard` protegen la navegación.
- `authInterceptor` añade `Authorization: Bearer <token>` y cierra sesión ante `401`.
- Servicios de `core/services` encapsulan la API; los modelos viven en `core/models`.

3. Tecnologías y requisitos

Capa	Tecnología	Uso
Frontend	Angular 21	SPA, rutas, formularios, signals
UI	Angular Material 21	Tablas, diálogos, menús, formularios
Backend	Node.js 20+	Entorno de ejecución
Servidor	Express 5	API REST y middlewares
Lenguaje	TypeScript	Tipado estricto en backend y frontend
Validación	Zod 3	DTO y variables de entorno
Base de datos	MongoDB 7+	Persistencia documental
ODM	Mongoose	Esquemas, índices y agregaciones

Capa	Tecnología	Uso
Autenticación	JWT + bcrypt	Sesión y hash de contraseñas
Logs	Pino + `pino-roll`	Diagnóstico técnico y rotación de archivos
Markdown	`marked`	Renderizado en visor y editor

Requisitos prácticos:

- Node.js 20.19+ o 22.12+ (el entorno validado usa Node 22).
- npm 10+; el frontend declara npm 11.6+.
- MongoDB local o remoto.
- Puerto `3000` para la API y `4200` para el frontend en desarrollo.

4. Estructura del repositorio

```
mi-drive/
├── backend/
│   ├── src/
│   │   ├── modules/          # auth, usuario, carpeta, archivo,
│   │   │                     # actividad, reporte, busqueda
│   │   ├── shared/           # config, container, middlewares, errors, utils
│   │   ├── scripts/seed.ts   # datos de demostración
│   │   ├── app.ts            # composición de Express
│   │   └── server.ts         # arranque y conexión a MongoDB
│   └── README.md
├── frontend/
│   ├── src/app/
│   │   ├── core/             # modelos, servicios, guards, interceptor, utils
│   │   ├── features/         # layout, mi-unidad, explorador, contextual,
│   │   │                     # editor, visor, papelera, perfil, admin, auth
│   │   ├── app.routes.ts
│   │   └── app.config.ts
│   └── README.md
├── docs/
│   ├── assets/               # capturas para los manuales
│   ├── pdf/                  # PDF generados
│   ├── manual-tecnico.md
│   └── manual-usuario.md
├── AGENTS.md                 # contexto local, no versionado
└── README.md
```

5. Modelo documental

MongoDB se organiza en cuatro colecciones principales. Las relaciones se representan con `ObjectId`; no se usan joins relacionales, sino referencias y agregaciones.

5.1 Colección `usuarios`

Campo	Tipo	Reglas
`_id`	ObjectId	Identificador
`nombre`	string	2–150 caracteres, `trim`
`correo`	string	Único, minúsculas, `trim`
`passwordHash`	string	bcrypt; `select: false`
`rol`	enum	`usuario` o `administrador`
`activo`	boolean	Controla el acceso
`createdAt`, `updatedAt`	Date	Timestamps automáticos

El `passwordHash` se elimina en `toJSON` y no se devuelve en las respuestas. El correo tiene índice único.

5.2 Colección `carpetas`

Campo	Tipo	Reglas
`nombre`	string	1–200 caracteres
`propietario`	ObjectId → usuario	Dueño del árbol
`creadoPor`	ObjectId → usuario	Atribución de autoría
`carpetaPadre`	ObjectId → carpeta o `null`	Jerarquía; `null` = raíz
`compartidoCon`	arreglo	`{ usuario, permiso, fechaCompartido }`
`enPapelera`	boolean	Eliminación lógica
`fechaEnvioPapelera`	Date o `null`	Momento del envío
`papeleraRaiz`	ObjectId o `null`	Raíz del envío a papelera
`createdAt`, `updatedAt`	Date	Timestamps

Restricción de unicidad parcial: no puede existir otra carpeta **activa** con el mismo `propietario`, `carpetaPadre` y `nombre`.

5.3 Colección `archivos`

Campo	Tipo	Reglas
`nombre`	string	1–300 caracteres
`tipo`	enum	`txt` o `md`

Campo	Tipo	Reglas
`contenido`	string	Hasta 10 000 000 de caracteres
`tamanio`	number	Bytes UTF-8 del contenido
`propietario`	ObjectId → usuario	Dueño del árbol
`creadoPor`	ObjectId → usuario	Autor de la creación
`carpetaPadre`	ObjectId → carpeta o `null`	Ubicación
`compartidoCon`	arreglo	Permiso directo por usuario
`enPapelera`, `fechaEnvioPapelera`, `papeleraRaiz`	—	Igual que carpetas
`createdAt`, `updatedAt`	Date	Timestamps

El contenido se almacena embebido en el documento. Es una decisión adecuada para el alcance académico y para archivos de texto pequeños o medianos: evita un sistema de archivos externo y simplifica la edición y la búsqueda.

> **Importante:** el campo persistido es `tamanio` (sin ñ). La API expone alias como `tamaño` y `tamañoTotal`; en agregaciones siempre se usa `$tamanio`.

5.4 Colección `actividades`

Campo	Tipo	Uso
`usuario`	ObjectId	Autor de la acción
`usuarioNombre`, `usuarioCorreo`	string	Snapshot histórico
`accion`	enum	Tipo de operación
`recursoTipo`	enum	`archivo`, `carpeta`, `usuario`, `sistema`
`recursoId`	ObjectId o `null`	Referencia polimórfica
`recursoNombre`	string o `null`	Snapshot del nombre
`metadata`	objeto	Datos adicionales de la operación
`createdAt`	Date	Fecha del registro

Los snapshots permiten que el historial conserve el nombre que tenía el usuario o el recurso en el momento de la acción, aunque después se renombre.

5.5 Relaciones e invariantes

```
Usuario 1 ──── N Carpeta        (propietario / creadoPor)
Carpeta 1 ──── N Carpeta        (carpetaPadre)
Carpeta 1 ──── N Archivo        (carpetaPadre)
Usuario N ──── N Carpeta/Archivo (compartidoCon)
Usuario 1 ──── N Actividad      (usuario)
```

Invariantes principales:

- Cada árbol de carpetas y archivos tiene **un único propietario**.
- Crear dentro de una carpeta compartida conserva como propietario al dueño del árbol.
- Mover un recurso a otro árbol transfiere la propiedad del recurso y, en carpetas, de todo su subárbol.
- La jerarquía se limita a una profundidad de seguridad de 20 niveles en las agregaciones de ancestros y descendientes.

5.6 Índices relevantes

- `usuarios.correo` único; índices por `rol` y `activo`.
- `carpetas`: `(propietario, carpetaPadre, enPapelera)`, `(carpetaPadre, enPapelera, nombre)`, `(compartidoCon.usuario, enPapelera)`.
- `archivos`: `(propietario, carpetaPadre, enPapelera)`, `(carpetaPadre, enPapelera)`, `(compartidoCon.usuario, enPapelera)`, `tipo`.
- `actividades`: `(usuario, createdAt)`, `(accion, createdAt)`, `(recursoTipo, recursoId, createdAt)` y `createdAt`.

6. Seguridad y control de acceso

6.1 Autenticación

1. El usuario envía correo y contraseña a `POST /api/auth/sign-in`.
2. El backend busca la cuenta, verifica que esté activa y compara con bcrypt.
3. Si es válida, firma un JWT con `id`, `nombre`, `correo` y `rol`.
4. El frontend guarda el token y el usuario en `localStorage` (`dms_token` y `dms_user`).
5. El interceptor añade `Authorization: Bearer <token>` a cada petición.

En cada petición protegida, `authenticate` verifica el JWT y **vuelve a cargar el usuario desde MongoDB**. Así, una cuenta desactivada pierde el acceso de inmediato y un cambio de rol se aplica sin esperar a que caduque el token.

- Token inválido o expirado → `401`.
- Usuario inexistente o inactivo → `401`.
- Sin token → `401`.

6.2 Contraseñas

- Se almacenan con bcrypt y un número de rondas configurable (`BCRYPT_ROUNDS`, 10 por defecto).
- Mínimo 6 caracteres al crear o cambiar.
- El cambio de contraseña exige la contraseña actual y que la nueva sea diferente.
- Solo el propio usuario puede cambiar su contraseña.

6.3 Roles

Rol	Capacidades
`usuario`	Opera sus recursos, recibe compartidos y edita su perfil
`administrador`	Todo lo anterior más gestión de usuarios, reportes y auditoría global

El middleware `authorize` restringe rutas por rol. En el frontend, `adminGuard` redirige a `/home` si el usuario no es administrador. El administrador conserva su menú normal y además ve el bloque de administración.

6.4 Permisos sobre recursos

Niveles admitidos: `lectura`, `escritura` y `comentar`.

Nivel	Lectura / descarga	Editar contenido	Renombrar	Compartir / papelera
Propietario	Sí	Sí	Sí	Sí
`escritura`	Sí	Sí	Sí	No
`lectura`	Sí	No	No	No
`comentar`	Sí	No	No	No

> En esta versión, `comentar` se comporta como acceso de lectura. No existe todavía un módulo de comentarios; el nivel queda reservado para una extensión futura.

Reglas de resolución:

- El propietario siempre tiene permiso de escritura.
- Un permiso directo sobre el archivo o la carpeta prevalece.
- Si no hay permiso directo, se hereda el del **ancestro más cercano** que tenga uno.
- Sin acceso al recurso se responde `404` para no revelar su existencia.
- Con acceso pero sin permiso para la acción se responde `403`.
- Solo el propietario puede compartir, mover o enviar a la papelera.
- No se puede compartir con uno mismo ni con usuarios inactivos.

- Compartir de nuevo con el mismo usuario actualiza su permiso.

6.5 Otras medidas

- CORS restringido por `CORS_ORIGIN`; en desarrollo debe incluir exactamente el origen usado, por ejemplo `http://localhost:4200` (no `127.0.0.1`).
- Validación de todos los cuerpos y parámetros con Zod.
- Límite de body HTTP de 16 MB.
- Sanitización de HTML de Angular al renderizar Markdown con `innerHTML`.
- El token en `localStorage` es aceptable para el alcance académico, pero una versión de producción debería valorar cookies `HttpOnly`, CSP y rotación de tokens.

7. Reglas de negocio

7.1 Carpetas y archivos

- El nombre es obligatorio y se guarda con `trim`.
- No se permiten duplicados activos en la misma ubicación y propietario.
- Los archivos incluyen la extensión en el nombre (`informe.md`).
- La extensión define el tipo (`txt` o `md`); no se aceptan otros formatos.
- El contenido máximo es de 10 millones de caracteres y el body HTTP de 16 MB.
- El tamaño se calcula en bytes UTF-8, no en cantidad de caracteres.
- No se puede crear ni mover dentro de una carpeta en papelera.
- No se puede mover una carpeta dentro de sí misma o de un descendiente.
- No se puede restaurar un recurso si su carpeta padre sigue en la papelera.

7.2 Papelera

- La eliminación es lógica: `enPapelera = true`.
- Al enviar una carpeta se marca todo su subárbol y sus archivos con la misma `papeleraRaiz`.
- En la papelera solo se muestran las raíces enviadas explícitamente, no cada hijo.
- Restaurar una carpeta restaura su subárbol completo.
- Un hijo eliminado por cascada no se restaura por separado; debe restaurarse la raíz.
- Solo el propietario puede enviar o restaurar.
- El repositorio incluye `eliminarPermanente`, pero **no hay ruta ni flujo de UI** para el borrado definitivo en esta entrega. La papelera es, por tanto, recuperable.

7.3 Usuarios

- El correo es único y se normaliza a minúsculas.
- El registro público crea siempre usuarios con rol `usuario`.

- Solo un administrador puede crear usuarios con otro rol, cambiarlo o activar/desactivar cuentas.
- Un administrador no puede desactivar su propia cuenta.
- Un administrador no puede cambiar su propio rol.
- La desactivación es lógica (`activo = false`); no se borra el documento.
- Un usuario desactivado no puede iniciar sesión ni usar un token previo.

7.4 Búsqueda

`GET /api/buscar?q=texto` busca por nombre, de forma parcial y sin distinguir mayúsculas, entre:

- recursos propios;
- recursos compartidos directamente;
- recursos dentro de carpetas compartidas y sus descendientes.

El límite por grupo es configurable y se acota a 20 en el servicio. El frontend usa la búsqueda de la barra superior y muestra carpetas y archivos agrupados.

8. API REST

8.1 Convenciones

- Base: `http://localhost:3000/api`.
- Formato: JSON UTF-8.
- Autenticación: `Authorization: Bearer <token>` salvo en registro, login y health.
- Fechas: ISO 8601.
- IDs: ObjectId de 24 caracteres hexadecimales.
- Errores: `{ "error": "mensaje", "requestId": "..." }` y, en validaciones, `{ "error": "Datos inválidos", "detalles": [...], "requestId": "..." }`.
- Correlación: cada respuesta incluye `X-Request-Id`; el mismo valor aparece en el log técnico de la petición.
- Listados paginados:
  - `GET /api/usuarios` → `{ usuarios, total, page, limit }`.
  - `GET /api/actividad` → `{ actividades, total, page, limit }`.

Códigos usados:

Código	Significado
`200`	Operación correcta
`201`	Recurso creado
`204`	Operación correcta sin cuerpo
`400`	Datos inválidos o regla de negocio incumplida

Código	Significado
`401`	No autenticado, token inválido o usuario inactivo
`403`	Autenticado sin permiso para la acción
`404`	Recurso inexistente o sin acceso
`409`	Conflicto de unicidad
`500`	Error interno

8.2 Endpoints

Salud

Método	Ruta	Acceso
GET	`/api/health`	Público

Autenticación

Método	Ruta	Acceso	Descripción
POST	`/api/auth/sign-in`	Público	Inicia sesión y devuelve token
POST	`/api/auth/sign-up`	Público	Registra un usuario `usuario`
GET	`/api/auth/me`	Autenticado	Devuelve el usuario de la sesión

Usuarios

Método	Ruta	Acceso	Descripción
GET	`/api/usuarios`	Admin	Lista paginada con filtros `rol`, `activo`
POST	`/api/usuarios`	Admin	Crea usuario
GET	`/api/usuarios/directorio`	Autenticado	Usuarios activos para compartir
GET	`/api/usuarios/:id`	Propio o admin	Detalle
PATCH	`/api/usuarios/:id`	Propio o admin	Nombre/correo; `rol` y `activo` solo admin
PATCH	`/api/usuarios/:id/password`	Propio	Cambia contraseña
DELETE	`/api/usuarios/:id`	Admin	Desactiva (soft delete)
POST	`/api/usuarios/:id/reactivar`	Admin	Reactiva

Carpetas

Método	Ruta	Descripción
POST	`/api/carpetas`	Crea carpeta
GET	`/api/carpetas`	Lista propias y compartidas
GET	`/api/carpetas/papelera`	Raíces propias en papelera
GET	`/api/carpetas/:id`	Detalle con `miPermiso`
PATCH	`/api/carpetas/:id/renombrar`	Renombra
PATCH	`/api/carpetas/:id/mover`	Mueve a otro padre o raíz
DELETE	`/api/carpetas/:id`	Envía a papelera en cascada
POST	`/api/carpetas/:id/restaurar`	Restaura subárbol
POST	`/api/carpetas/:id/compartir`	Comparte o actualiza permiso
DELETE	`/api/carpetas/:id/compartir/:usuarioId`	Quita compartido

Archivos

Método	Ruta	Descripción
POST	`/api/archivos`	Crea archivo
GET	`/api/archivos`	Lista por carpeta, tipo, búsqueda y paginación
GET	`/api/archivos/compartidos`	Archivos compartidos con el usuario
GET	`/api/archivos/papelera`	Raíces propias en papelera
GET	`/api/archivos/:id`	Detalle con contenido y `miPermiso`
POST	`/api/archivos/:id/abrir`	Registra la apertura en el visor
GET	`/api/archivos/:id/descargar`	Descarga como `text/plain`
PUT	`/api/archivos/:id/contenido`	Actualiza contenido y tamaño
PATCH	`/api/archivos/:id/renombrar`	Renombra
PATCH	`/api/archivos/:id/mover`	Mueve a otra carpeta o raíz
DELETE	`/api/archivos/:id`	Envía a papelera
POST	`/api/archivos/:id/restaurar`	Restaura
POST	`/api/archivos/:id/compartir`	Comparte o actualiza permiso
DELETE	`/api/archivos/:id/compartir/:usuarioId`	Quita compartido

Actividad

Método	Ruta	Acceso	Descripción
GET	`/api/actividad/mi-historial`	Autenticado	Historial propio
GET	`/api/actividad`	Admin	Historial global paginado
GET	`/api/actividad/recurso/:tipo/:id`	Según acceso	Historial de un recurso

Filtros disponibles: `usuarioId`, `accion`, `recursoTipo`, `recursoId`, `desde`, `hasta`, `page`, `limit` (máximo 200).

Reportes

Método	Ruta	Acceso	Descripción
GET	`/api/reportes/archivos/por-tipo`	Admin	Totales y tamaño por extensión
GET	`/api/reportes/archivos/por-rango`	Admin	Archivos creados en un rango
GET	`/api/reportes/archivos/mas-operaciones`	Admin	Archivos con más actividad
GET	`/api/reportes/usuarios/top-archivos`	Admin	Top de usuarios por archivos
GET	`/api/reportes/usuarios/mas-compartidos`	Admin	Usuarios que más comparten
GET	`/api/reportes/compartidos/mas-compartidos`	Admin	Archivos más compartidos
GET	`/api/reportes/compartidos/por-usuario/:usuarioId`	Propio o admin	Recursos que comparte
GET	`/api/reportes/compartidos/recibidos/:usuarioId`	Propio o admin	Recursos que recibió
GET	`/api/reportes/actividad/mi-rango`	Autenticado	Actividad propia por fechas
GET	`/api/reportes/actividad/por-tipo`	Autenticado	Conteo propio por acción
GET	`/api/reportes/actividad/por-periodo`	Autenticado	Conteo propio por día/mes/año
GET	`/api/reportes/admin/resumen`	Admin	KPIs globales del dashboard
GET	`/api/reportes/admin/archivos/por-tipo`	Admin	Distribución global por tipo

Método	Ruta	Acceso	Descripción
GET	`/api/reportes/admin/actividad/por-tipo`	Admin	Conteo global por acción
GET	`/api/reportes/admin/actividad/por-periodo`	Admin	Conteo global por período

Búsqueda

Método	Ruta	Acceso	Descripción
GET	`/api/buscar?q=texto&limite=8`	Autenticado	Búsqueda global de archivos y carpetas

9. Actividad y auditoría

9.1 Actividad de negocio

La colección `actividades` registra acciones funcionales y es la fuente de los reportes y de la auditoría visible en el panel administrativo.

Acciones registradas por el enum `AccionActividad`:

Valor	Significado
`crear`	Alta de archivo, carpeta o usuario
`renombrar`	Cambio de nombre
`mover`	Cambio de ubicación
`actualizar_Contenido`	Edición del contenido de un archivo
`abrir`	Apertura de un archivo desde el visor
`compartir`	Alta o cambio de permiso
`quitar_compartido`	Retiro de un permiso
`enviar_papelera`	Eliminación lógica
`restaurar`	Recuperación desde papelera
`eliminar_permanente`	Borrado definitivo desde la papelera
`login`	Inicio de sesión
`registro`	Registro público
`desactivar`	Desactivación de usuario
`reactivar`	Reactivación de usuario

Valor	Significado
`actualizar`	Cambio de datos o contraseña

Características:

- El registro es **best-effort**: si falla la auditoría, el error se envía al logger técnico pero no se revierte la operación principal.
- Se guardan snapshots de nombre y correo del usuario y del nombre del recurso.
- La metadata varía según la acción; por ejemplo, un movimiento guarda carpeta de origen y destino, y una compartición guarda usuario destino y permiso.
- El administrador puede filtrar por usuario, acción, tipo de recurso y fechas.
- Los reportes usan agregaciones de MongoDB para contar por acción y por período.

9.2 Logs técnicos

Los logs técnicos no se guardan en `actividades` ni se muestran en el frontend. Su objetivo es diagnosticar fallos operativos.

- Implementación: Pino, `pino-http` y `pino-roll`.
- Salida: consola legible en desarrollo; JSON por stdout y archivo en producción.
- Archivos: `logs/app.FECHA.N.log`, con rotación diaria o al alcanzar 10 MB.
- Retención: `LOG_MAX_DAYS` archivos rotados (14 por defecto).
- Cada petición recibe un UUID en `X-Request-Id` y se registra con método, ruta sin query, estado, duración, IP, user-agent y usuario autenticado.
- Los `4xx` se registran como `warn`; los `5xx`, como `error` con stack.
- El serializador de errores solo conserva tipo, mensaje, stack, código y estado; evita propiedades arbitrarias como `body`.
- Nunca se registran bodies, contraseñas, tokens, cookies ni contenido de archivos.
- La carpeta `logs/` está excluida de Git y no se expone mediante la API.

10. Rendimiento y decisiones técnicas

10.1 Resolución de permisos en lote

El principal riesgo de un modelo jerárquico es consultar los ancestros una vez por archivo. Para evitarlo:

- Los permisos directos se resuelven en memoria.
- Las carpetas sin permiso directo se agrupan y sus ancestros se obtienen con una sola agregación `$graphLookup`.
- Se busca en memoria el ancestro más cercano con permiso.
- Existe una caché por operación (`Map`) para no repetir la resolución.

10.2 Operaciones masivas en papelera

Enviar una carpeta a la papelera no recorre los hijos uno por uno. Se obtienen los IDs del subárbol y se ejecutan dos actualizaciones masivas: una para carpetas y otra para archivos. La restauración usa el mismo enfoque.

10.3 Índices y agregaciones

- Las consultas por ubicación, propietario y papelera usan índices compuestos.
- Los reportes usan `$group`, `$lookup`, `$unionWith` y `$graphLookup` según el caso.
- La búsqueda usa expresiones regulares escapadas para evitar inyección de patrones.
- Los listados de archivos excluyen `contenido` salvo que se solicite explícitamente.

10.4 Limitaciones conocidas

- No hay paginación uniforme en todos los listados de carpetas y archivos; algunos devuelven el conjunto completo de la carpeta.
- El contenido embebido no es adecuado para archivos binarios o muy grandes.
- No existe borrado permanente expuesto ni módulo de comentarios.
- Las rutas `/recientes` y `/compartidos` aparecen en el menú, pero aún no tienen vista propia; `/home` es una página mínima.
- Los scripts de regresión y E2E usados durante el desarrollo son locales y no están versionados.
- No se incluye configuración de despliegue, respaldo automático ni observabilidad.

11. Instalación y operación

11.1 Backend

```
cd backend
npm install
cp .env.example .env
```

Variables:

```
PORT=3000
NODE_ENV=dev
MONGO_URI=mongodb://localhost:27017/dms
JWT_SECRET=un_secreto_de_al_menos_32_caracteres
JWT_EXPIRES_IN=7d
BCRYPT_ROUNDS=10
CORS_ORIGIN=http://localhost:4200
LOG_LEVEL=info
LOG_DIR=logs
LOG_MAX_DAYS=14
LOG_TO_FILE=true
```

```
npm run seed   # destructivo: borra y recrea datos
npm run dev    # API en http://localhost:3000
npx tsc --noEmit
```

11.2 Frontend

```
cd frontend
npm install
npm start      # http://localhost:4200
npx ng build
```

11.3 Verificación mínima

```
curl http://localhost:3000/api/health
```

Respuesta esperada:

```
{"status":"ok","timestamp":"2026-10-20T..."}
```

11.4 Datos de demostración

Rol	Correo	Contraseña
Administrador	`ana@dms.local`	`password123`
Usuario	`carlos@dms.local`	`password123`
Usuario	`lucia@dms.local`	`password123`

El seed crea 3 usuarios, 15 carpetas y 50 archivos (algunos en papelera), además de compartidos y actividad. Es **destructivo** y cambia los IDs en cada ejecución.

12. Guía de mantenimiento

12.1 Añadir o modificar un endpoint

1. Define o ajusta el esquema Zod en `*.dto.ts`.
2. Añade la ruta con `authenticate`, `authorize` y los `validate*` necesarios.
3. Implementa el caso de uso en el servicio.
4. Usa el repositorio para el acceso a datos; no consultes Mongoose desde el controller.
5. Registra la actividad si la operación modifica estado.
6. Actualiza el contenedor si hay nuevas dependencias.
7. Verifica con `npx tsc --noEmit` y una prueba manual o automatizada.

12.2 Añadir una vista Angular

1. Crea el componente standalone en `features/`.
2. Usa signals para estado local y `inject()` para dependencias.
3. Añade la ruta lazy en `app.routes.ts`.
4. Protege con `authGuard` o `adminGuard` según corresponda.
5. Reutiliza `ExploradorCarpetaComponent` y `MenuContextualComponent` cuando aplique.
6. Ejecuta `npx ng build`.

12.3 Gotchas del proyecto

- `tamanio` **sin ñ** en MongoDB y agregaciones; la API puede exponer alias con ñ.
- El parser de plantillas de Angular no acepta `tamañoTotal` como identificador. Usa helpers con acceso por corchetes: `item['tamañoTotal']`.
- No apliques `min-height` externo al textarea con `cdkTextareaAutosize`; el directive lo interpreta como altura de línea. Usa `cdkAutosizeMinRows` y `cdkAutosizeMaxRows`.
- El seed con `updateMany` y pipeline requiere `{ updatePipeline: true }`.
- El CORS debe coincidir exactamente con el origen del navegador.
- No commitear `.env`, `AGENTS.md` ni `prompts/`.

13. Verificación realizada

Durante la implementación se comprobó:

- `npx ng build` sin errores ni advertencias.
- `npx tsc --noEmit` sin errores.
- Regresión de API: 54/54, 59/59 y 19/19 casos.
- E2E de administración: 38/38.
- E2E de vista lista/cuadrícula: 15/15.
- Navegación sin excepciones JavaScript en las vistas principales.

Estas verificaciones fueron manuales o con scripts locales, por lo que no forman parte del repositorio versionado. Para una entrega reproducible se recomienda incorporarlas como pruebas automatizadas del proyecto.

14. Glosario

Término	Definición
DMS	Document Management System; sistema de gestión documental
Soft delete	Marcado lógico como eliminado sin borrar el documento
`papeleraRaiz`	ID del recurso que originó un envío en cascada a la papelera

Término	Definición
Permiso efectivo	Permiso final tras considerar dueño, share directo y herencia
Snapshot	Copia del nombre o correo en el momento de registrar actividad
JWT	Token firmado usado para autenticar peticiones
SPA	Single Page Application; aplicación de una sola página
